

1 ViT và bài học về inductive bias (Vision Transformers and Inductive Bias)

PHỤ THUỘC C/13/01-dong-cnn (ba giả định mà convolution cài sẵn); C/13/02-tu-chuoi-toi-attention (attention, mã hoá vị trí, chỉ phí bậc hai).

TÓM TẮT

Mục này dùng *Vision Transformer* làm thí nghiệm sạch nhất cho một câu hỏi trung tâm của cả cây: cài sẵn giả định vào mô hình thì được gì và mất gì. ViT vứt bỏ hai giả định mà convolution cài sẵn, và cái giá lộ ra rất rõ ràng dưới dạng *lượng dữ liệu cần có*. Đọc xong bạn quyết định được khi nào nên dùng ViT, khi nào nên dùng CNN, khi nào nên lai. Bạn cũng đọc được mã nguồn của một hệ detection trong mười phút, nhờ cách tách *backbone*, *neck* và *head*. Nếu chỉ nhớ một điều: *inductive bias là một khoản vay trả trước cho dữ liệu* — bạn hoặc trả bằng giả định, hoặc trả bằng mẫu, và ViT chỉ đơn giản chọn cách trả thứ hai.

KIẾN THỨC CẦN CÓ TRƯỚC

- **inductive bias** — tập giả định mà mô hình mang sẵn trước khi nhìn dữ liệu, ví dụ “pixel gần nhau thì liên quan”; nó thu hẹp không gian giả thuyết nên học nhanh hơn với ít dữ liệu, nhưng nếu giả định sai thì nó chặn luôn nghiệm đúng.
- **patch** — ô vuông nhỏ được cắt ra từ ảnh, thường 16×16 pixel, rồi làm phẳng thành một vector để coi như một token; đây là cách ViT biến ảnh thành chuỗi.
- **đẳng biến và bất biến (equivariance vs invariance)** — đẳng biến tịnh tiến nghĩa là dịch ảnh thì bản đồ đặc trưng dịch theo đúng như vậy; bất biến nghĩa là dịch ảnh thì đầu ra *không đổi*. Convolution cho đẳng biến, còn phép gộp toàn cục ở cuối mới tạo ra bất biến.
- **pretraining và fine-tuning** — huấn luyện trước trên một tập rất lớn để học đặc trưng chung, rồi huấn luyện tiếp trên tập nhỏ của bài toán đích; đây là cách phổ biến nhất để bù lượng dữ liệu mà một mô hình ít giả định đòi hỏi.
- **chưng cất (distillation)** — huấn luyện mô hình học sinh bắt chước đầu ra của một mô hình thầy đã tốt, thay vì chỉ học từ nhãn cứng; nó chuyển được cả các giả định ngầm của thầy sang trò.
- **backbone, neck, head** — ba tầng chức năng của một hệ thị giác: backbone trích đặc trưng từ ảnh, neck hợp nhất đặc trưng ở nhiều tỉ lệ, head sinh đầu ra theo đúng bài toán (nhãn lớp, hộp, mặt nạ).
- **FPN (feature pyramid network)** — một kiểu neck ghép đặc trưng độ phân giải cao nhưng ngữ nghĩa yếu ở tầng nông với đặc trưng ngữ nghĩa mạnh nhưng thô ở tầng sâu, để phát hiện được cả vật nhỏ lẫn vật lớn.

A. Đặt vấn đề

Suốt chương 13 ta lặp lại một câu hỏi: nên cài sẵn bao nhiêu giả định vào mô hình? Câu hỏi đó rất khó trả lời bằng lý luận, vì hai vế của nó nằm ở hai đơn vị khác nhau. Một bên là “mô hình biết trước điều gì”; bên kia là “có bao nhiêu dữ liệu”. *Vision Transformer* (ViT) [1] biến câu hỏi đó thành một thí nghiệm đo được: lấy một kiến trúc gần như không có giả định thị giác nào, huấn luyện nó ở nhiều quy mô dữ liệu khác nhau, và xem điểm giao cắt nằm ở đâu.

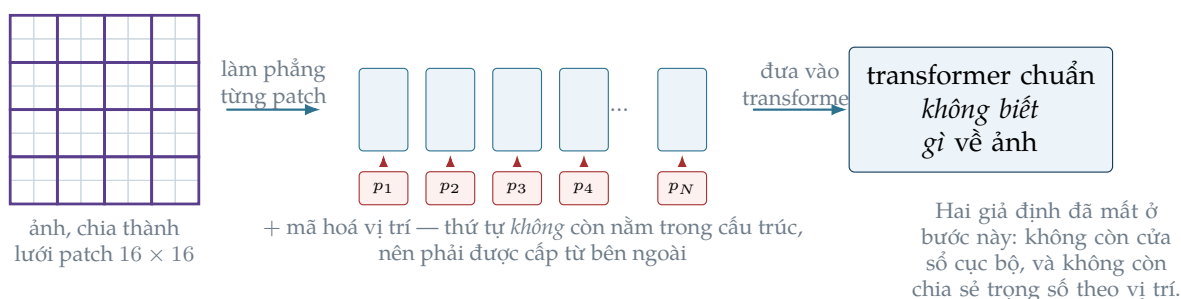
Kết quả là câu trả lời sạch nhất mà ngành có: với dữ liệu ít, ViT thua CNN rõ rệt; với dữ liệu rất nhiều, ViT vượt. Giá trị cụ thể của mục này là biến quan sát đó thành một quy tắc quyết định dùng được. Quy tắc ấy chạy trên ba dữ kiện: bạn có bao nhiêu ảnh có nhãn, bạn có mô hình pretrain nào, và ràng buộc độ phân giải của bạn là gì. Mục này cũng cho bạn một cách đọc mã nguồn nhanh cho bất kỳ hệ thị giác nào.

↔ **tiền đề** B/10 — trực đánh đổi giả định–dữ liệu được phát biểu ở chương thiết kế kiến trúc; mục này là phép đo thực nghiệm cho chính trực đó

B. ViT bỏ đi cái gì

ViT cắt ảnh thành các patch 16×16 , làm phẳng mỗi patch thành một vectơ, cộng mã hoá vị trí, rồi đưa cả dãy vào một transformer chuẩn. Phép biến đổi đó vứt bỏ hai trong ba giả định của convolution nêu ở C/13/01.

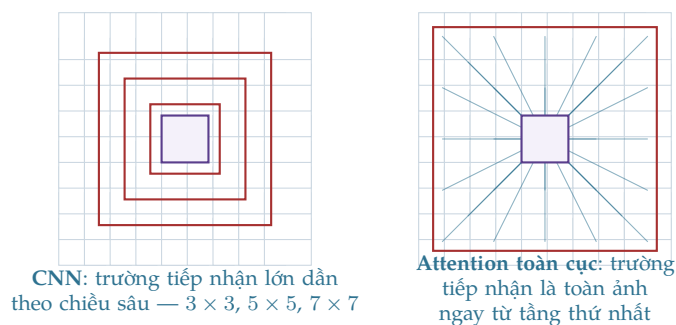
- **Tính cục bộ biến mất.** Ngay từ tầng đầu tiên, mọi patch nói chuyện với mọi patch. Không có gì trong kiến trúc nói rằng hai patch cạnh nhau liên quan nhiều hơn hai patch ở hai góc ảnh — mô hình phải *học* điều đó từ dữ liệu.
- **Đẳng biến tịnh tiến biến mất.** Convolution dùng chung một bộ trọng số cho mọi vị trí nên dịch ảnh thì đặc trưng dịch theo. ViT gán một mã vị trí riêng cho từng ô, nên cùng một vật ở hai vị trí khác nhau đi qua hai đường tính toán hơi khác nhau.
- **Phân cấp cũng gần như biến mất** ở bản ViT gốc: mọi tầng chạy ở cùng một độ phân giải token, không có chuỗi giảm dần độ phân giải như CNN. Đây là giả định thứ ba, và chính nó sẽ được các biến thể phân cấp lấy lại đầu tiên.



Hình 1. Ba bước biến một ảnh thành thứ mà transformer xử lý được, và chỗ inductive bias rơi ra. Việc cắt patch vẫn giữ lại một chút tính cục bộ: 256 điểm ảnh trong một ô bị gộp cứng. Nhưng *quan hệ giữa* các patch thì không còn được mã hoá ở đâu trong kiến trúc. Nó phải được cấp bằng một bảng vị trí học được. Đó là lý do đổi độ phân giải đầu vào mà quên nội suy bằng này làm mô hình xuống cấp lạng lẽ: bạn vừa cấp sai đúng cái thông tin mà kiến trúc không tự có.

Hình 1 cho thấy điều mà câu “ViT coi ảnh như một chuỗi” còn giấu: chuỗi đó không có thứ tự tự nhiên, và toàn bộ hiểu biết về không gian hai chiều bị dồn vào một bảng tham số phụ.

Cần nói ngay một điều để tránh hiểu quá đà: *ViT không phải không có giả định*. Bản thân việc cắt thành patch là một giả định cục bộ khá mạnh — 256 pixel trong một ô bị gộp thành một token và không bao giờ được xét riêng nữa. Ngoài ra, các phân tích attention cho thấy ở những tầng đầu, nhiều đầu tự học ra hành vi gần như cục bộ, tức mô hình khám phá lại một phần giả định đã bị vứt. Phát biểu đúng là: ViT giảm mạnh lượng giả định cài sẵn, chứ không xoá sạch.

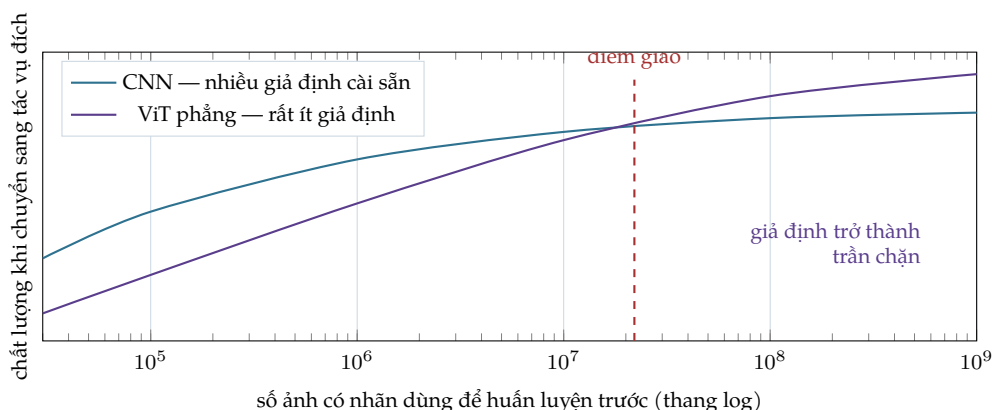


Hình 2. Hai cách phủ sóng thông tin. CNN mở rộng trường tiếp nhận dần dần và do đó *cài sẵn* rằng quan hệ gần quan trọng hơn quan hệ xa; attention toàn cục nối mọi vị trí ngay lập tức và không cài giả định nào về khoảng cách, đổi lại phải học quan hệ đó từ dữ liệu và trả chi phí bậc hai. Swin và các biến thể cửa sổ nằm giữa hai thái cực này.

Hình 2 là cách gọn nhất để thấy cả ưu điểm lẫn cái giá: bên trái, thông tin xa cần nhiều tầng nhưng mỗi tầng rẻ và có sẵn tiên nghiệm đúng cho ảnh tự nhiên; bên phải, thông tin xa có ngay nhưng phải trả bằng T^2 cặp quan hệ và bằng việc mô hình không được cho biết cặp nào đáng quan tâm.

C. Cái giá hiện ra dưới dạng lượng dữ liệu

Kết quả gốc của ViT rất dễ tóm tắt: huấn luyện từ đầu trên ImageNet-1k thì nó thua các ResNet cùng hạng; pretrain trên một tập lớn hơn nhiều bậc rồi fine-tune thì nó vượt [1]. Đây là minh chứng trực tiếp cho trục đánh đổi giả định–dữ liệu: giả định mà bạn không cài vào kiến trúc thì phải mua bằng mẫu.



Hình 3. Trục đánh đổi giả định–dữ liệu, vẽ định tính. Ở phía trái, các giả định mà convolution cài sẵn là món quà miễn phí: chúng thu hẹp không gian giả thuyết đúng chỗ, nên CNN học được từ ít mẫu. Ở phía phải, chính các giả định ấy thành trần — chúng cấm mô hình biểu diễn những quan hệ mà dữ liệu đã đủ để dạy nó. Điều quan trọng nhất trong hình là có một điểm giao chứ không phải vị trí của nó: vị trí phụ thuộc tác vụ, recipe và quy mô mô hình, và ba cách trả nợ giả định ở mục C đều là cách kéo đường tím sang trái.

Hình 3 biến một câu định tính thành một hình có thể tranh luận được: câu hỏi “CNN hay ViT” thực chất là câu hỏi “tôi đang đứng ở đâu trên trục hoành”.

Điều làm câu chuyện thú vị là có ít nhất ba cách trả khoản nợ đó, và chúng trả bằng ba loại tiền khác nhau.

- **Trả bằng dữ liệu** — pretrain trên tập cực lớn, rồi fine-tune. Hiệu quả nhất về chất lượng, nhưng chỉ khả thi nếu bạn có tập đó hoặc có checkpoint của người khác; và nó đẩy toàn bộ chi phí sang một pha huấn luyện mà bạn không kiểm soát.
- **Trả bằng chính quy hoá và chứng cất** — DeiT cho thấy với augmentation mạnh, lịch học dài và một thầy CNN để chứng cất, ViT huấn luyện được trên riêng ImageNet-1k [2]. Đáng chú ý là thầy CNN chuyển sang trò không chỉ nhận mà cả *giả định* của mình — đây là một cách cài inductive bias qua cửa sau.
- **Trả bằng kiến trúc** — lắp lại một phần giả định đã bỏ, tức các biến thể phân cấp ở mục D. Rẻ nhất về dữ liệu, nhưng đánh mất sự thuần khiết và một phần khả năng mở rộng theo quy mô.

↪ **hệ quả của** C/18 — pretraining tự giám sát là cách rẻ nhất để trả khoản nợ giả định; các backbone mạnh nhất hiện nay đều đến từ hướng này.

↪ **cùng nguyên lý với** C/12/04 — augmentation mạnh và chứng cất đều bơm thêm giả định vào mô hình, chỉ khác đường đi.

D. Phản ứng thứ nhất: ViT phân cấp

ViT phẳng có hai vấn đề thực dụng ngoài chuyện dữ liệu: nó chỉ sinh ra đặc trưng ở một tỉ lệ duy nhất, trong khi detection và segmentation cần nhiều tỉ lệ; và chi phí bậc hai chặn đường tới độ phân giải cao, như đã tính ở C/13/02. Swin giải cả hai bằng một thay đổi: giới hạn attention trong các cửa sổ cố định rồi dịch cửa sổ ở tầng kế tiếp để thông tin rò qua ranh giới, đồng thời gộp patch dần để tạo kim tự tháp đặc trưng [3]. Chi phí trở thành tuyến tính theo số token, và đầu ra có đúng dạng mà FPN cần. PVT đi theo hướng gần giống nhưng giảm số khoá và giá trị thay vì giới hạn cửa sổ. Còn lại ghép đơn giản nhất là thay phép cắt patch bằng vài tầng convolution làm “stem”. Cách này được biết là làm giai đoạn huấn luyện đầu ổn định hơn hẳn. Đây là kinh nghiệm thực hành rất phổ biến, không phải một kết quả tôi kiểm chứng lại.

Cái giá thì cụ thể: cửa sổ tạo lại một dạng “đường đi dài” cho quan hệ xa, đúng thứ mà attention sinh ra để xoá; và kiến trúc nhiều giai đoạn khó mở rộng theo quy mô một cách trơn tru hơn so với một chồng khối đồng nhất.

E. Phản ứng thứ hai: ConvNeXt, và vì sao nó là phản chứng quan trọng

Hướng ngược lại không sửa transformer mà sửa cách so sánh. ConvNeXt giữ nguyên convolution rồi lần lượt áp từng thành phần recipe của transformer, đo sau mỗi bước, và kết quả là CNN bất kíp [4]. Kết luận đã nêu ở C/13/01 và nhắc lại ở đây vì nó thay đổi cách đọc chính mục này: phần lớn khoảng cách được gán cho “attention thay convolution” thực ra thuộc về công thức huấn luyện. Nói cách khác, thí nghiệm ViT vẫn đúng về trục giả định–dữ liệu, nhưng độ lớn của hiệu ứng kiến trúc đã bị phóng đại trong cách trình bày đương thời.

F. Ba dạng tổ chức và ba lớp bài toán

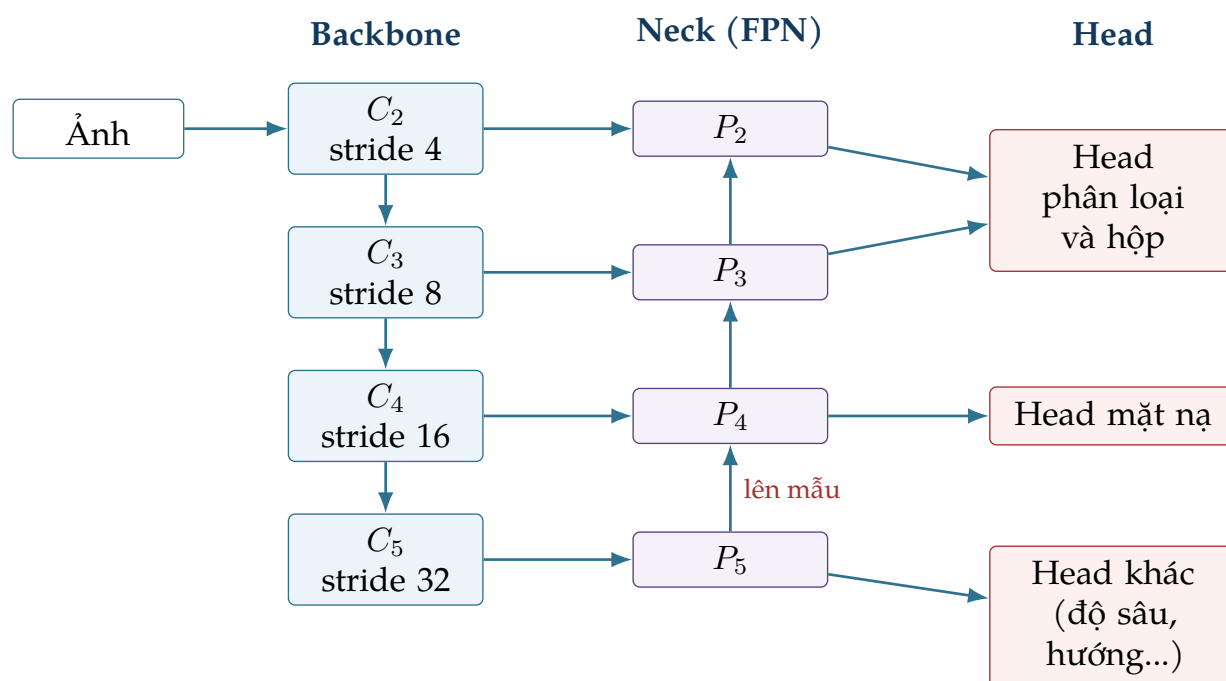
Chọn sai dạng tổ chức là lỗi thiết kế cơ bản, không phải lỗi tinh chỉnh. Không có siêu tham số nào cứu được một encoder-only đang bị bất sinh chuỗi tự hồi quy.

Bảng 1. Ba dạng tổ chức của một mô hình dựa trên transformer, và lớp bài toán tương ứng.

Dạng	Cơ chế	Hợp với bài toán nào	Hỏng khi nào
Encoder-only	Mọi token thấy mọi token, một lượt	Hiểu và dự đoán từ toàn bộ đầu vào: phân loại, dự đoán dài, trích đặc trưng cho hệ khác	Cần sinh chuỗi có độ dài chưa biết trước, hoặc cần sinh tự hồi quy
Decoder-only	Attention có mặt nạ nhân quả, sinh từng token dựa trên các token đã sinh	Sinh tự hồi quy: ngôn ngữ, chuỗi hành động, mã hoá ảnh thành token rồi sinh	Suy luận chậm vì tuần tự; lãng phí khi đầu ra là một nhân duy nhất
Encoder-decoder	Encoder đọc đầu vào, decoder sinh đầu ra và tra cứu encoder bằng cross-attention	Biến đổi chuỗi sang chuỗi; trong thị giác là DETR với tập truy vấn học được [5]	Nhiều tham số và nhiều đường nối hơn; hội tụ chậm hơn ở bài toán nhỏ

G. Tách backbone, neck, head — cách đọc code nhanh nhất

Ba khái niệm này không phải lý thuyết mà là quy ước kỹ thuật, và chúng có giá trị thực dụng rất cao: gần như mọi repo detection và segmentation hiện đại đều tổ chức theo đúng ba khối này, nên nắm được chúng là đọc được cấu trúc một repo lạ trong mười phút thay vì một ngày.



Hình 4. Ba khối chức năng của một hệ thị giác dày. Backbone sinh đặc trưng ở nhiều stride; neck hợp nhất chúng theo chiều từ sâu về nông để mọi tỉ lệ đều có ngữ nghĩa mạnh; head chuyển đặc trưng thành đầu ra của bài toán. Thay backbone là thao tác rẻ nhất và hay làm nhất; thay neck ảnh hưởng chủ yếu tới vật nhỏ; thay head là đổi bài toán.

Hình 4 cũng giải thích vì sao ViT phải khó lắp vào hệ detection: nó chỉ sinh ra một cột duy nhất chứ không sinh ra bốn mức C_2 tới C_5 , nên hoặc phải chế thêm một neck đặc biệt, hoặc phải chuyển sang biến thể phân cấp. Đây là một ràng buộc kỹ thuật rất thực tế và thường quyết định lựa chọn backbone hơn cả điểm số ImageNet.

↔ **tiền đề** C/14 — các head của detection và segmentation giả định có sẵn bốn mức đặc trưng, nên dạng đầu ra của backbone quyết định head nào lắp được

H. Giới hạn và trường hợp hỏng

Bốn chỗ hỏng đáng nhớ, xếp theo tần suất gặp trong thực hành.

Thứ nhất, *ViT huấn luyện từ đầu trên tập nhỏ*. Đây là cách phổ biến nhất để kết luận sai rằng “transformer không hợp bài toán của tôi”. Nếu bạn có vài nghìn ảnh, kết luận đúng là bạn không đủ dữ liệu để trả khoản nợ giả định, chứ không phải kiến trúc kém.

Thứ hai, *kích thước patch so với kích thước đối tượng*. Một patch 16×16 nghiền mọi đối tượng nhỏ hơn 16 pixel thành một phần của một token duy nhất. Với ảnh camera gắn xe, người đi bộ ở xa chỉ chiếm mười pixel. Đây không phải chi tiết nhỏ mà là giới hạn cứng. Nó giải thích vì sao các hệ dự đoán dày dùng patch nhỏ hoặc dùng conv stem.

Thứ ba, *mã hoá vị trí khi đổi độ phân giải*, đã nói ở mục trước và nhắc lại vì nó thực sự hay xảy ra.

Thứ tư, *mất đẳng biến tịnh tiến là có thật và đo được*. Cùng một vật ở hai vị trí có thể cho đặc trưng hơi khác nhau. Trong phần lớn trường hợp, augmentation cắt ngẫu nhiên đã bù đủ. Nhưng ở các bài toán đo lường chính xác như ước lượng tư thế hay đo khoảng cách, đây là một nguồn sai số hệ thống đáng kiểm tra chứ không nên giả định là không có.

Về độ tin cậy: quan hệ “ít dữ liệu thì CNN thắng, nhiều dữ liệu thì ViT thắng” là kết quả đã công bố và được lặp lại rộng rãi; còn mức dữ liệu cụ thể ở điểm giao cắt phụ thuộc tác vụ, recipe và quy mô mô hình tới mức tôi không nêu một con số nào ở đây.

CẠM BẤY

Cạm bẫy tư duy lớn nhất của mục này là đọc “ViT không có inductive bias” theo nghĩa đen rồi suy ra “ViT là mô hình trung lập, không giả định gì”. Không đúng. Phép cắt patch là một giả định cục bộ rất mạnh. Mã hoá vị trí là một giả định về cách biểu diễn không gian. Và cách chọn dữ liệu pretrain là một giả định khổng lồ về thế giới mà mô hình sẽ gặp. Phát biểu chính xác là ViT *dịch chuyển* giả định từ kiến trúc sang dữ liệu và recipe — và giả định nằm trong dữ liệu thì khó nhìn thấy hơn nhiều so với giả định nằm trong kiến trúc.

CẦN NHỚ

Inductive bias là khoản vay trả trước cho dữ liệu: giả định bạn không cài vào kiến trúc thì phải mua bằng mẫu, bằng chứng cất, hoặc bằng augmentation. Khi chọn giữa CNN, ViT và các biến thể lai, câu hỏi đầu tiên không phải “cái nào mạnh hơn” mà là “tôi có bao nhiêu dữ liệu và bao nhiêu tiền nghiệm đáng tin về bài toán này”. [T2]

I. Kết nối

Mục này khép lại chương 13 bằng cách trả lời câu hỏi mở ở C/13/01-dong-cnn: ba giả định của convolution đáng giá bao nhiêu, đo bằng lượng dữ liệu cần để thay thế chúng. Nó dựa hoàn toàn vào cơ chế attention ở C/13/02-tu-chuoi-toi-attention, và cách tách backbone-neck-head ở mục G là tiền đề trực tiếp cho C/14-bai-toan-cv-loi, nơi các head cụ thể được thiết kế. Về phía dữ liệu, việc trả nợ giả định bằng pretraining là nội dung của C/18-pretraining-va-foundation, còn việc trả bằng augmentation đã nằm ở C/12/04-tong-quat-hoa-va-chan-doan. Cuối cùng, bài học ConvNeXt nối sang E/24-thi-nghiem-va-ablation, chương duy nhất trong cây dạy cách không bị một bảng so sánh đánh lừa.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. ViT bỏ đi hai giả định nào của convolution, và mỗi giả định bị bỏ gây ra hậu quả đo được nào?
2. Vì sao nói ViT vẫn có inductive bias, dù nó bỏ tính cục bộ và đẳng biến tịnh tiến?
3. Ba cách bù lại lượng dữ liệu mà ViT đòi hỏi là gì, và mỗi cách trả giá bằng loại tài nguyên nào?
4. Vì sao Swin lắp vào một hệ detection dễ hơn ViT phẳng — câu trả lời nằm ở khối nào trong sơ đồ backbone-neck-head?
5. Nếu ConvNeXt cho thấy recipe quan trọng hơn kiến trúc, điều đó thay đổi cách bạn đọc một bảng so sánh backbone thế nào?
6. Bạn cần phát hiện người đi bộ cách xa 80 mét, chỉ chiếm khoảng 12 pixel chiều cao. Vì sao ViT patch 16 là lựa chọn tệ, và hai cách sửa là gì?
7. Trong hệ của bạn, thay backbone, thay neck và thay head có tác động khác nhau thế nào — và vì sao thứ tự thử nên là backbone trước?

Nguồn

- [1] A. Dosovitskiy, L. Beyer, A. Kolesnikov, et al., “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale,” in *ICLR*, 2021.
- [2] H. Touvron, M. Cord, M. Douze, et al., “Training Data-Efficient Image Transformers and Distillation Through Attention,” in *ICML*, 2021.
- [3] Z. Liu, Y. Lin, Y. Cao, et al., “Swin Transformer: Hierarchical Vision Transformer Using Shifted Windows,” in *ICCV*, 2021.
- [4] Z. Liu, H. Mao, C.-Y. Wu, et al., “A ConvNet for the 2020s,” in *CVPR*, 2022.
- [5] N. Carion, F. Massa, G. Synnaeve, et al., “End-to-End Object Detection with Transformers,” in *ECCV*, 2020.